Practical 1 — Substitution and Transposition Cipher

A. Caesar Cipher

Remember: Encryption shifts letters forward; decryption shifts letters backward.

```
def encrypt(text, s):
    result = ""
    for ch in text:
        if ch.isalpha():
            base = ord('A') if ch.isupper() else ord('a')
            result += chr((ord(ch) - base + s) % 26 + base)
        else:
            result += ch
    return result

def decrypt(text, s):
    return encrypt(text, -s)

text = input("Enter text: ")
s = 3
cipher = encrypt(text, s)

print("Encrypted:", cipher)
print("Decrypted:", decrypt(cipher, s))
```

B. Rail Fence Cipher

Remember: Write letters in a zigzag pattern and read row by row.

```
def encrypt(text, depth):
    rows = [""] * depth
    row, direction = 0, 1

    for ch in text:
        rows[row] += ch
        if row == 0:
            direction = 1
        elif row == depth - 1:
            direction = -1
        row += direction

    return "".join(rows)

text = input("Enter message: ")
depth = int(input("Enter depth: "))
print("Encrypted:", encrypt(text, depth))
```

Practical 2 — RSA Encryption and Decryption

Remember: RSA uses a public key to encrypt and a private key to decrypt.

```
p = 61
q = 53
n = p * q
phi = (p - 1) * (q - 1)
e = 17
d = 2753

text = input("Enter text: ")
encrypted = [pow(ord(ch), e, n) for ch in text]
decrypted = ''.join(chr(pow(x, d, n)) for x in encrypted)

print("Encrypted:", encrypted)
print("Decrypted:", decrypted)
```

Practical 3 — Message Authentication Code (MAC)

Remember: Generate a MAC, then compare it with the received MAC.

```
import hmac
import hashlib

key = b"secretkey"
message = input("Enter message: ")

mac = hmac.new(
   key, message.encode(), hashlib.sha256
).hexdigest()

print("Generated MAC:", mac)

received = input("Enter MAC for verification: ")

if hmac.compare_digest(mac, received):
   print("Message is Authentic")
else:
   print("Message has been Tampered")
```

Practical 4 — Digital Signature

Remember: The sender signs with a private key; the receiver verifies with a public key.

This is a simplified version of your file's RSA-based demonstration.

```
import hashlib

p, q = 61, 53
n = p * q
e, d = 17, 2753

message = input("Enter message: ")

h = int(hashlib.sha256(message.encode()).hexdigest(), 16)
signature = pow(h, d, n)

print("Digital Signature:", signature)

verified = pow(signature, e, n)

if verified == h % n:
   print("Signature Verified")
else:
   print("Verification Failed")
```

Practical 5 — Diffie–Hellman Key Exchange

Remember: Two users calculate a common secret key without directly sending that key.

```
Xa = int(input("Enter Xa: "))
Xb = int(input("Enter Xb: "))

p = int(input("Enter prime p: "))
a = int(input("Enter primitive root a: "))

Ya = pow(a, Xa, p)
Yb = pow(a, Xb, p)

Ka = pow(Yb, Xa, p)
Kb = pow(Ya, Xb, p)

if Ka == Kb:
   print("Transmission successful")
else:
   print("Transmission failed")
```

Practical 6 — IPsec simulation

Aim: Check message integrity and authenticity using HMAC.

Memory trick: Message → HMAC → Send → Verify

```
import hashlib
import hmac

key = b"ipsec_secret_key"
message = input("Enter message to send: ")

packet = {
   "data": message,
   "hash": hmac.new(
      key, message.encode(), hashlib.sha256
   ).hexdigest()
}

print("Sending packet:", packet)

received_data = packet["data"]
received_hash = packet["hash"]

verify = hmac.new(
   key, received_data.encode(), hashlib.sha256
).hexdigest()

if verify == received_hash:
   print("IPsec Verification Successful!")
   print("Message is Authentic and Untampered")
   print("Received Message:", received_data)
else:
   print("IPsec Verification Failed!")
```

Easy explanation:

1. Import hashlib and hmac.
2. Create a shared secret key.
3. Generate an HMAC for the message.
4. Simulate sending the packet.
5. Generate HMAC again at the receiver.
6. Compare both values.

Practical 7 — SSL/TLS secure communication

A. Generate certificate

```
from cryptography import x509
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import rsa

private_key = rsa.generate_private_key(
   public_exponent=65537,
   key_size=2048
)

print("Private key generated")
```

B. Server — receive message and reply

```
import socket
import ssl

HOST = "127.0.0.1"
PORT = 5000

context = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
context.load_cert_chain("server.crt", "server.key")

server_socket = socket.socket()
server_socket.bind((HOST, PORT))
server_socket.listen(5)

with context.wrap_socket(
   server_socket, server_side=True
) as server:
   connection, address = server.accept()

   with connection:
      data = connection.recv(1024).decode()
      print("Received:", data)

      connection.send(
         "Hello from SSL/TLS Server!".encode()
      )
```

C. Client — send message and receive reply

```
import socket
import ssl

HOST = "127.0.0.1"
PORT = 5000

context = ssl.create_default_context()
context.check_hostname = False
context.verify_mode = ssl.CERT_NONE

sock = socket.socket()

with context.wrap_socket(
   sock, server_hostname=HOST
) as client:
   client.connect((HOST, PORT))
   client.send("Hello".encode())

   response = client.recv(1024).decode()
   print("Server Response:", response)
```

How to run: Start the server first in one terminal, then the client in another. Generate the certificate files before starting the server.

Viva: SSL/TLS provides secure communication through encryption, authentication and integrity. The journal disables certificate verification for its local demonstration; that is not safe for real-world connections.

Practical 8 — Intrusion Detection System (IDS)

Aim: Detect suspicious network traffic, such as port scanning and ICMP flooding.

```
from scapy.all import sniff, IP, TCP, ICMP
from collections import defaultdict
import time

connections = defaultdict(list)

def detect_attack(packet):
    if not packet.haslayer(IP):
        return

    src = packet[IP].src
    dst = packet[IP].dst
    now = time.time()

    if packet.haslayer(TCP):
        key = (src, dst)
        connections[key].append(
            (packet[TCP].dport, now)
        )

        connections[key] = [
            x for x in connections[key]
            if now - x[1] <= 10
        ]

        ports = {x[0] for x in connections[key]}

        if len(ports) >= 10:
            print("ALERT: Port Scan Detected!")

    if packet.haslayer(ICMP):
        key = ("ICMP", src)
        connections[key].append(now)

        connections[key] = [
            t for t in connections[key]
            if now - t <= 10
        ]

        if len(connections[key]) >= 20:
            print("ALERT: ICMP Flood Detected!")

print("Monitoring network traffic...")
sniff(prn=detect_attack, store=False)
```

Easy explanation:

1. Scapy captures network packets.
2. The program checks the IP layer.
3. It counts TCP destination ports.
4. Ten different ports within ten seconds trigger a port-scan alert.
5. Twenty ICMP packets within ten seconds trigger an ICMP-flood alert.

Practical 9 — Malware Analysis Tool

Aim: Inspect a file, calculate its SHA-256 hash and warn about potentially executable file types.

```
import os
import hashlib

path = input("Enter file path: ")

if not os.path.isfile(path):
   print("File not found!")
   exit()

print("File Name:", os.path.basename(path))
print("File Size:", os.path.getsize(path), "bytes")

sha256 = hashlib.sha256()

with open(path, "rb") as file:
   while True:
      data = file.read(4096)
      if not data:
         break
      sha256.update(data)

print("SHA-256:", sha256.hexdigest())

extension = os.path.splitext(path)[1].lower()

suspicious = [
   ".exe", ".dll", ".bat", ".cmd",
   ".vbs", ".js", ".scr", ".ps1"
]

if extension in suspicious:
   print("Warning: Further analysis recommended")
else:
   print("No suspicious executable extension detected")

print("Analysis completed successfully.")
```

Practical 10 — Firewall Rule Simulation

Aim: Allow or block network traffic based on IP address and port.

```
ALLOWED_IP = "192.168.1.10"
BLOCKED_PORTS = [21, 23, 445]

def firewall_check(source_ip, port):
    print("\nChecking Network Traffic...")
    print("Source IP:", source_ip)
    print("Destination Port:", port)

    if source_ip != ALLOWED_IP:
        print("Result: BLOCKED")
        print("Reason: Unauthorized IP")
        return

    if port in BLOCKED_PORTS:
        print("Result: BLOCKED")
        print("Reason: Port is blocked")
        return

    print("Result: ALLOWED")

firewall_check("192.168.1.10", 80)
firewall_check("192.168.1.20", 80)
firewall_check("192.168.1.10", 23)
```

Cyber & Information Security Practical Viva — Complete Question Bank

Practical 1: Substitution and Transposition Ciphers

1. What is cryptography?

Cryptography is the technique of protecting information by converting readable text into an unreadable form.

2. What is plaintext?

Plaintext is the original, readable message.

3. What is ciphertext?

Ciphertext is the encrypted, unreadable message.

4. What is encryption?

Encryption converts plaintext into ciphertext using an algorithm and a key.

5. What is decryption?

Decryption converts ciphertext back into plaintext.

6. What is a substitution cipher?

It replaces each letter with another letter according to a rule.

7. What is a transposition cipher?

It changes the positions of letters without replacing the letters themselves.

8. What is a Caesar Cipher?

It shifts each letter by a fixed number of positions in the alphabet.

9. What shift value is used in your Caesar Cipher program?

The shift value is 3.

10. What is the difference between encryption and decryption in Caesar Cipher?

Encryption shifts letters forward; decryption shifts them backward.

11. What are ord() and chr() in Python?

ord() returns a character's Unicode number, and chr() converts a number into a character.

12. What is the Rail Fence Cipher?

It is a transposition cipher that writes letters in a zigzag pattern and reads them row by row.

13. What is depth in Rail Fence Cipher?

Depth is the number of rows used to write the zigzag pattern.

14. Is Caesar Cipher secure today?

No. It has a very small key space and can be broken easily.

Practical 2: RSA Encryption and Decryption

1. What is RSA?

RSA is a public-key cryptographic algorithm used for encryption and digital signatures.

2. Who developed RSA?

Ron Rivest, Adi Shamir and Leonard Adleman.

3. What type of cryptography does RSA use?

Asymmetric cryptography.

4. What are the two RSA keys?

Public key and private key.

5. What is the public key used for?

In the basic RSA encryption model, it encrypts data for the key owner.

6. What is the private key used for?

It decrypts data encrypted with the corresponding public key.

7. Why are p and q used?

They are prime numbers used to calculate the RSA modulus.

8. What is n in RSA?

$n=p\times q$, the modulus used in both keys.

9. What is Euler's totient value in this program?

$\phi(n)=(p-1)(q-1)$, since p and q are distinct primes.

10. What does pow(byte, e, n) do?

It calculates the byte raised to exponent e, modulo n.

11. What is the main advantage of RSA?

It allows public-key cryptography without requiring the sender to know the receiver's private key.

12. Is RSA faster than AES?

Generally, no. AES is much faster for bulk data encryption.

Practical 3: Message Authentication Code (MAC)

1. What is MAC?

MAC stands for Message Authentication Code. It helps verify a message's integrity and authenticity using a secret key.

2. What is HMAC?

HMAC stands for Hash-based Message Authentication Code. It combines a secret key with a hash function.

3. Which Python modules are used?

hmac and hashlib.

4. Which hash algorithm is used in your program?

SHA-256.

5. What is a hash function?

A hash function converts input data into a fixed-length digest.

6. What is message integrity?

It means the message has not been changed without detection.

7. Why is a secret key used?

It helps ensure that only someone who knows the shared key can generate a valid MAC.

8. Why is compare_digest() used?

It compares authentication values in a way designed to reduce timing leaks.

9. What happens if the MAC values differ?

Verification fails, indicating that the message or its authentication value may have been changed or is invalid.

10. Does a MAC provide confidentiality?

No. It provides integrity and authentication, not encryption.

Practical 4: Digital Signature

1. What is a digital signature?

A digital signature is a cryptographic value used to verify a message's integrity and the signer's authenticity.

2. Which keys are used in a digital signature?

The sender signs with a private key, and the receiver verifies with the corresponding public key.

3. What is the purpose of a digital signature?

To provide authenticity, integrity and support for non-repudiation.

4. What is non-repudiation?

It means providing evidence that makes it difficult for a signer to deny signing a message.

5. Why is hashing used?

It produces a fixed-length digest of the message to be signed.

6. Which hash algorithm is used in your program?

SHA-256.

7. What happens if the message changes?

The message hash changes, and signature verification should fail.

8. How is a digital signature different from encryption?

Encryption protects confidentiality; a digital signature verifies authenticity and integrity.

9. What does pow(signature, e, n) do in your example?

It uses the RSA public exponent to perform the simplified signature verification calculation.

10. Is your practical a complete production-ready digital signature system?

No. It demonstrates the basic concept. Real systems use standardized signature schemes and secure hash encoding.

Practical 5: Diffie–Hellman Key Exchange

1. What is Diffie–Hellman?

It is a cryptographic method that allows two parties to establish a shared secret over a public channel.

2. Does Diffie–Hellman directly encrypt messages?

No. It establishes a shared secret that can be used to derive encryption keys.

3. What are p and a in your program?

p is a prime number, and a is a primitive root modulo p.

4. What are Xa and Xb?

They are the private values chosen by the two participants.

5. What are Ya and Yb?

They are the public values calculated by the participants.

6. What are Ka and Kb?

They are the shared secret values independently calculated by each participant.

7. Why should Ka and Kb be equal?

Both calculations produce the same shared secret when the parameters and calculations are correct.

8. What is the main advantage of Diffie–Hellman?

It allows two parties to establish a shared secret without transmitting that secret directly.

9. What is a man-in-the-middle attack?

An attacker secretly intercepts or alters communication between two parties.

10. How can Diffie–Hellman be protected against a man-in-the-middle attack?

By authenticating the participants, for example using digital certificates and signatures.

Practical 6: IPsec Simulation

1. What is IPsec?

IPsec stands for Internet Protocol Security. It protects IP communications using security protocols.

2. What are the main IPsec protocols?

Authentication Header (AH) and Encapsulating Security Payload (ESP).

3. What is HMAC used for in your program?

It checks message integrity and authenticity using a shared secret key.

4. What is the purpose of packet in your program?

It stores the message data and its HMAC value.

5. What happens at the receiver's side?

The receiver calculates the HMAC again and compares it with the received value.

6. What happens if the packet is modified?

The HMAC values should differ, and verification should fail.

7. Does your Python program configure real IPsec?

No. It simulates an integrity and authentication check; it does not create a real IPsec tunnel or encrypt network traffic.

8. What is the difference between AH and ESP?

AH provides integrity and authentication, while ESP can provide confidentiality through encryption and can also provide integrity and authentication.

9. What are the two IPsec modes?

Transport mode and tunnel mode.

10. Where is IPsec commonly used?

In VPNs and secure communication between networks or devices.

Practical 7: SSL/TLS Secure Communication

1. What is SSL?

SSL stands for Secure Sockets Layer. It is an older protocol for securing network communication.

2. What is TLS?

TLS stands for Transport Layer Security. It is the modern successor to SSL.

3. Why is TLS used?

It provides encryption, authentication and integrity for communication between clients and servers.

4. What is a client?

A client is the program or device that requests a service from a server.

5. What is a server?

A server listens for client connections and provides a service or response.

6. What is an SSL/TLS certificate?

It is a digital certificate that helps establish the identity of a server.

7. What are server.crt and server.key?

server.crt contains the certificate, while server.key contains the server's private key.

8. What is a socket?

A socket is an endpoint used for communication between programs over a network.

9. What is the purpose of port 5000 in your practical?

It is the port on which the local server listens for client connections.

10. What does recv(1024) do?

It receives up to 1024 bytes of data.

11. What does encode() do?

It converts a string into bytes for transmission.

12. What does decode() do?

It converts bytes back into a string.

13. Why must you start the server before the client?

The server must be listening before the client attempts to connect.

14. Why is certificate verification disabled in your client code?

It is disabled for the local demonstration. In real applications, certificate verification should normally remain enabled.

15. What is a TLS handshake?

It is the process in which the client and server negotiate security settings, authenticate as required, and establish session keys.

Practical 8: Intrusion Detection System (IDS)

1. What is IDS?

IDS stands for Intrusion Detection System. It monitors activities and detects possible security threats.

2. What are the main types of IDS?

Network-based IDS (NIDS) and host-based IDS (HIDS).

3. What is Scapy?

Scapy is a Python library used to create, send, capture and analyze network packets.

4. What does sniff() do?

It captures network packets for analysis.

5. What is a port scan?

A port scan checks multiple network ports to discover available services.

6. What is an ICMP flood?

It is an attack involving a large volume of ICMP packets that may overwhelm a target.

7. What is ICMP?

ICMP stands for Internet Control Message Protocol. It is used for network diagnostic and error messages.

8. What is TCP?

TCP stands for Transmission Control Protocol. It provides reliable, connection-oriented data delivery.

9. Why is time.time() used?

It records the current time so the program can count packets within a time window.

10. What threshold does your program use for port scanning?

Ten unique destination ports within ten seconds.

11. What threshold does your program use for an ICMP flood?

Twenty ICMP packets within ten seconds from the tracked source.

12. What is a false positive in IDS?

It occurs when normal or legitimate activity is incorrectly identified as suspicious.

13. Does IDS automatically block attacks?

Not necessarily. A traditional IDS detects and alerts; an Intrusion Prevention System (IPS) can take blocking action.

14. Why might your program require administrator privileges?

Packet capture often requires elevated permissions on the operating system.

15. What does store=False mean in Scapy?

It prevents captured packets from being stored in Scapy's packet list.

Practical 9: Malware Analysis Tool

1. What is malware?

Malware is malicious software designed to harm systems, steal data or perform unauthorized activities.

2. What is malware analysis?

It is the process of examining suspicious software to understand its behavior and potential risks.

3. Which Python modules are used in your program?

os and hashlib.

4. What does os.path.isfile() do?

It checks whether the given path points to a file.

5. What does os.path.getsize() do?

It returns the file size in bytes.

6. What is SHA-256?

SHA-256 is a cryptographic hash algorithm that produces a 256-bit digest.

7. What is a file hash?

It is a fixed-length value calculated from file contents and used to help identify or compare files.

8. Why is the file opened using "rb"?

r means read, and b means binary mode. This allows the program to read the raw file contents.

9. Why does the program read 4096 bytes at a time?

To process the file in manageable chunks rather than loading the entire file into memory.

10. What does hexdigest() do?

It returns the hash digest as a hexadecimal string.

11. Which file extensions does your program flag?

.exe, .dll, .bat, .cmd, .vbs, .js, .scr and .ps1.

12. Does a suspicious extension prove that a file is malware?

No. Legitimate programs can use these extensions, and malware can use other extensions.

13. What is static malware analysis?

It examines a file without executing it.

14. What is dynamic malware analysis?

It observes a program's behavior while it runs in a controlled environment.

15. How can malware infections be prevented?

Use updated security software, install security updates, avoid suspicious attachments and maintain backups.

Practical 10: Firewall Rule Simulation

1. What is a firewall?

A firewall monitors and filters network traffic according to security rules.

2. What are the main types of firewalls?

Packet-filtering, stateful inspection, proxy/application-level and next-generation firewalls.

3. What is an IP address?

An IP address identifies a network interface for communication over an IP network.

4. What is a port number?

A port number identifies a service or application endpoint on a device.

5. What is the purpose of ALLOWED_IP?

It specifies the IP address permitted by the program's rule.

6. What is the purpose of BLOCKED_PORTS?

It stores the port numbers that the program must block.

7. Which ports are blocked in your program?

Ports 21, 23 and 445.

8. What are these ports commonly associated with?

- Port 21 — FTP control
- Port 23 — Telnet
- Port 445 — SMB

9. What happens when an unauthorized IP address is entered?

The program prints BLOCKED and returns without checking the port further.

10. Why is return used?

It exits the function immediately.

11. What happens when the IP is allowed and the port is not blocked?

The program prints ALLOWED.

12. Is your program a real firewall?

No. It simulates firewall decisions in Python; it does not filter actual network packets.

13. What is the principle of least privilege?

Give users and systems only the permissions they need to perform their tasks.

14. What is the default-deny principle?

Block traffic unless a rule explicitly permits it.

15. Why are firewalls important?

They help prevent unauthorized access and reduce exposure to network threats.